# Trip Planner

Application overview • architecture • component structure • October 2026

## What it does

Trip Planner is an AI-powered travel planning web app. A signed-in user describes a trip and the app uses Claude to generate a day-by-day itinerary, with a map of each day's stops, local events, the best places to eat nearby, and all their flight and hotel confirmation codes in one place. Any trip can be downloaded as a PDF for use offline.

- **Plan a trip** — a five-step questionnaire: destination (type-to-search over 213 countries and territories, e.g. Puerto Rico), cities and dates (up to 14 days); flights (optional); where the traveler is staying, with an optional confirmation code; and travel style.
- **Background generation** — the trip is saved as *pending* and the server generates it with Claude, using flight times to keep arrival days light. The user sees an animated progress screen and can leave; the itinerary appears on the dashboard when ready.
- **Dashboard and calendar** — trip cards with a live status (*Draft*, *Pending*, *Ready*, *Failed* with Retry), and a month-view **trips calendar** with Today, Previous trip and Next trip.
- **Itinerary view** — per-day map, stops, tips and hotel, plus three sections: **Local Events**; **Food Spots** (top 12 places per city with “Load 10 more”, pinned on a map with distance from the hotel and a “Closest to homebase” sort); and **Reservations** (flights and hotels with tap-to-copy confirmation codes).
- **Offline PDF** — reservations and codes, a map and stops for every day, 10 local events and 10 food spots per city.
- **Accounts and PWA** — sign in with email/password or Google, profile with avatar; installable on a phone's home screen, with offline caching of the app shell and map tiles.

## Technology

**Frontend** — a single-page app in `src/`

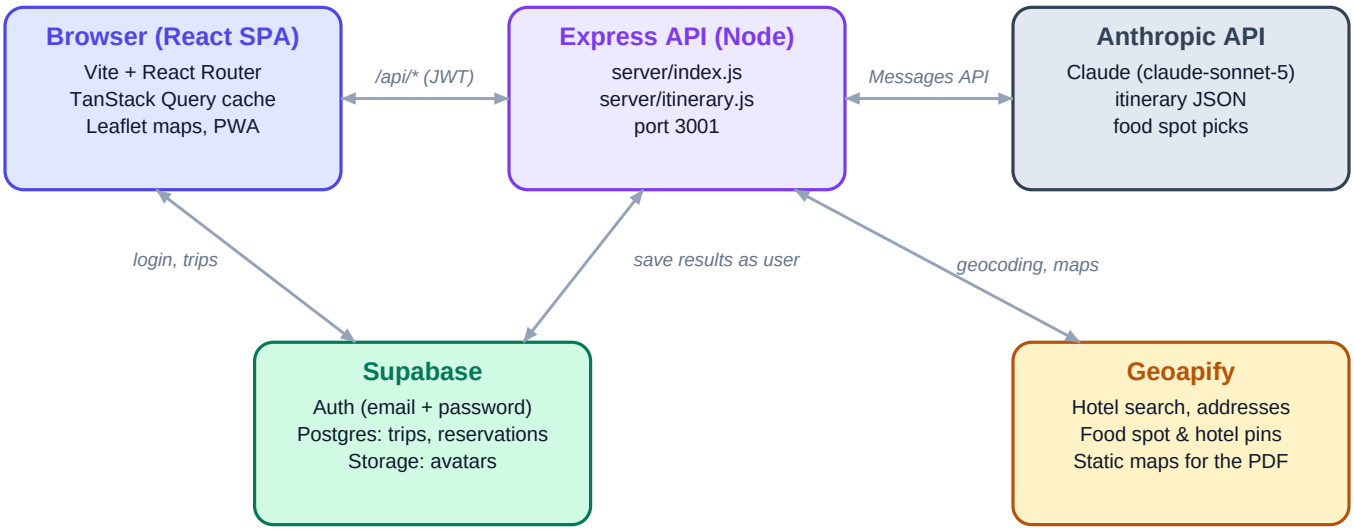
| | |
|---|---|
| **React 18** | UI, function components and hooks |
| **Vite 5** | Dev server and production build; compiles Tailwind through PostCSS |
| **React Router 6** | Routes: `/`, `/new`, `/drafts/:id`, `/trip/:id`, `/calendar`, `/profile` |
| **TanStack Query 5** | Fetching, caching and saving trips; polls every 4 s while a trip is pending |
| **Tailwind CSS 3** | Styling; one shared theme in `utils/theme.js` |
| **Framer Motion** | Animations: page transitions, modals, loading-screen globe and plane |
| **Leaflet + react-leaflet** | Interactive maps on OpenStreetMap tiles |
| **lucide-react, vite-plugin-pwa** | Icons; service worker and web-app manifest |

### Backend and services

| | |
|---|---|
| **Node.js + Express 5** | Server in `server/`: the API (itinerary, food spots, PDF, hotel search, address validation) and, in production, the built app, on one port. Holds the secret API keys. |
| **Anthropic SDK** | Calls Claude (`claude-sonnet-5`): streams the itinerary as JSON, and picks food spots at low effort for speed |
| **Supabase** | Authentication (email/password and Google), Postgres database (`trips`, `members`) and file storage (avatars). Used directly by the browser, and by the server acting as the signed-in user. |
| **Geoapify** | Hotel search, address validation, map coordinates for food spots and hotels, and static map images for the PDF |
| **pdfkit** | Builds the offline PDF on the server |
| **Docker + Railway** | Multi-stage image running one Node server; deployed on Railway, which rebuilds on every push (GitHub Actions also builds the image in CI) |

## Architecture

The browser talks to Supabase directly for sign-in and reading or writing trips. Anything that needs a secret key goes through the Express server.



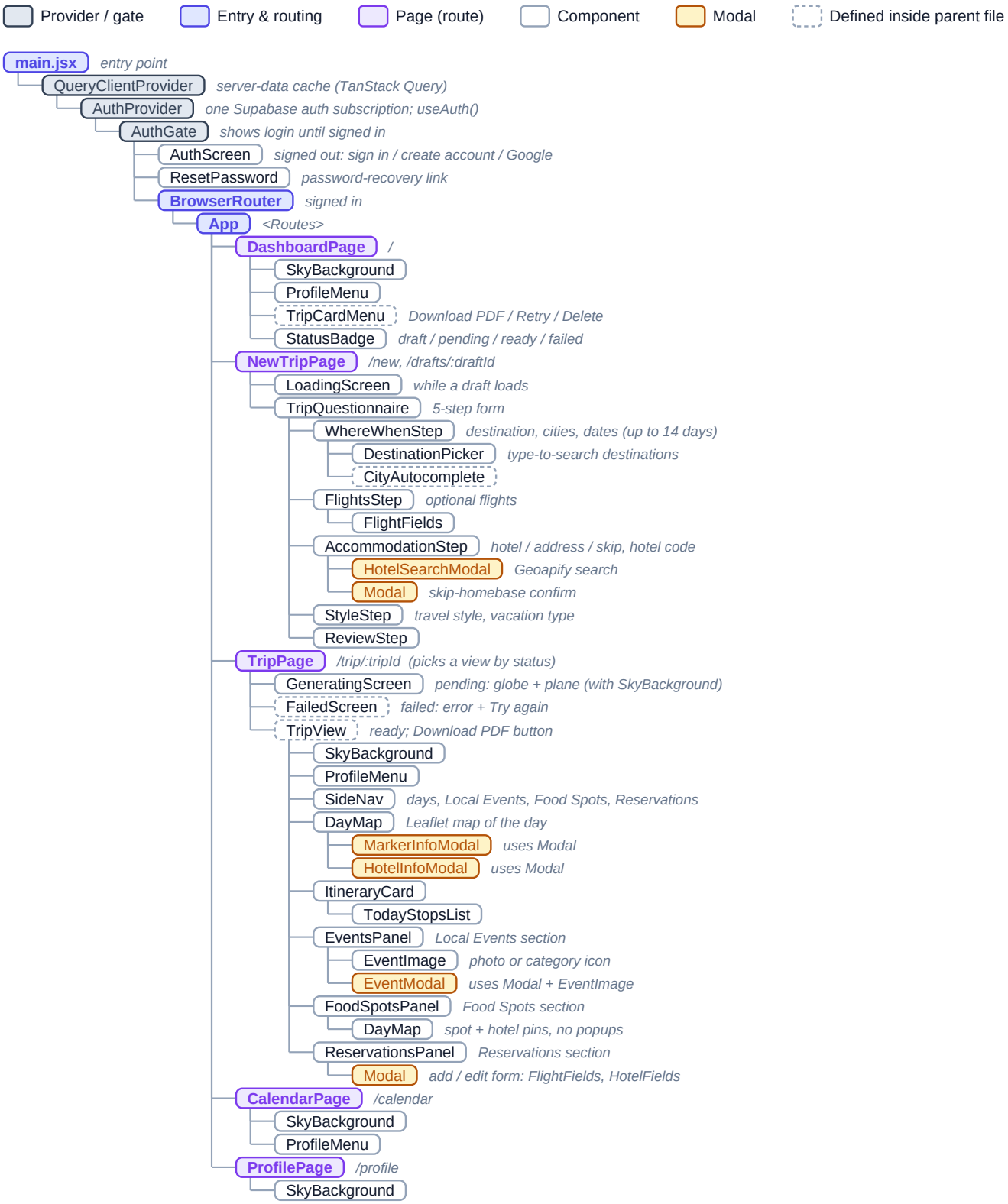

## How an itinerary is generated

1. The user finishes the questionnaire. The browser saves the trip to Supabase with status *pending* and its form answers.
2. The browser calls `POST /api/generate-itinerary` with only the trip id and the user's login token, then opens `/trip/:id`, which shows the progress screen.
3. The server reads the trip's answers as that user, replies right away, and in the background runs three things in parallel: the itinerary from Claude (the prompt includes flight routes and times, never confirmation codes); 12 food spots per city, each located on the map with Geoapify; and each city's hotel location.
4. The server merges the results into the trip with status *ready* (or *failed* with the error), re-reading the row first so edits made meanwhile aren't overwritten. A trip still pending after 10 minutes is treated as failed.
5. The progress screen and the dashboard poll the trip every 4 seconds and switch to the itinerary as soon as it is ready, even if the user left and came back.

Everything is stored in the trip's `itinerary_data` JSON (itinerary, questionnaire answers with reservations, status, food spots, hotel locations), so no database schema change was needed. A mock mode (`VITE_USE_MOCK_DATA=true`) skips the API and returns sample data for UI work.

## Server modules

| | |
|---|---|
| **index.js** | Express routes (trips read and written as the signed-in user), the 14-day trip limit, and serving the built app in production |
| **itinerary.js** | Builds the itinerary prompt, calls Claude, parses the JSON |
| **foodSpots.js** | Asks Claude for a city's best places to eat, then locates them on the map |
| **geoapify.js** | Geocoding, hotel lookup and static maps, rate-limited to the free tier (4 requests/s) |
| **pdf.js** | Lays out the offline PDF |

## Component tree

How the React components nest at runtime, starting from `src/main.jsx`. Grey italic notes give the route or the condition under which a branch renders. Shared pieces (`SkyBackground`, `ProfileMenu`, `Modal`) appear under each parent that uses them.